PROJECT PROPOSAL

NetCode Lab: Prediction, Interpolation and Redundancy for Real-Time Multiplayer Games over UDP

Computer Networks

1. Project Title, Group Numbers, Names and IDs

Project Title: NetCode Lab: Evaluating Client-Side Prediction, Entity Interpolation and Input Redundancy for Real-Time Multiplayer Games over UDP in a Routed Topology.

Group Members:

Name	Student ID	Primary Responsibility
Hatim Mustafa	24K-0673	Network topology, routing, netem, packet evidence and experiment automation
Fuzail Raza	24K-0699	Authoritative server, protocol and TCP baseline
Zaid Ahmed	24K-0500	Game client, client-side netcode and analysis

2. Proposed Project Description

NetCode Lab is a two-player top-down real-time arena game designed to study how common multiplayer networking techniques affect responsiveness and state consistency under controlled network conditions. The game will use an authoritative server and communicate over a custom binary UDP protocol. The networking environment will be implemented using Linux network namespaces, two routers and four subnets, allowing delay, jitter and packet loss to be introduced in a controlled and repeatable manner.

The project will compare a simple networked game against progressively more advanced client-side techniques. The main research question is how much client-side prediction, entity interpolation and input redundancy improve perceived responsiveness and state consistency under controlled delay and packet loss, and what bandwidth costs they introduce.

Functional Features

1. Authoritative multiplayer game: A two-player top-down arena game in which the server maintains the authoritative game state while clients send player inputs and receive state snapshots.

2. Custom UDP game protocol: A binary protocol with protocol ID, message type, sequence number, acknowledgement, acknowledgement bitfield and tick information. It supports JOIN, JOIN_ACK, INPUT, SNAPSHOT, EVENT and LEAVE messages.

3. Client-side prediction: The client immediately applies local player inputs instead of waiting for the server, then reconciles its state when an authoritative snapshot arrives by replaying unacknowledged inputs.

4. Entity interpolation: Remote entities are rendered slightly in the past and interpolated between received snapshots to produce smoother movement under network delay and packet loss.

5. Input redundancy: Each INPUT packet contains the most recent N inputs so that recent inputs can still reach the server when individual packets are lost.

6. Delta-compressed snapshots: Server snapshots are compressed against the last acknowledged snapshot to reduce the amount of state data transmitted.

7. Reliable event delivery: Join, leave and score events use a small acknowledged channel because these events cannot simply be replaced by newer state.

8. TCP baseline: The same game will also be implemented over TCP so its behaviour under packet loss can be compared with the UDP implementation, particularly head-of-line blocking.

9. Runtime netcode configurations: The client will support four progressively improved configurations: C0 naive, C1 prediction, C2 prediction plus interpolation, and C3 prediction plus interpolation, redundancy and delta compression.

Network Architecture

The system will run on a single Ubuntu laptop using Linux network namespaces connected by virtual Ethernet pairs. Two namespaces will act as routers, R1 and R2, with IP forwarding enabled. Client A will reach the server through both routers, while Client B will be attached to R2. The router-to-router link will be impaired using tc netem, allowing controlled delay, jitter and packet loss.

Node	IP Address	Role
Client A	10.0.1.10/24	Player 1 client
R1	10.0.1.1/24, 10.0.12.1/30	Gateway and first router
R2	10.0.12.2/30, 10.0.2.1/24, 10.0.3.1/24	Second router and gateways
Client B	10.0.2.10/24	Player 2 client
Game Server	10.0.3.10/24	Authoritative game server

Experimental Evaluation

Experiments will use deterministic inputs, seeded network impairments and repeated 60-second runs. Each condition will be run three times and reported using mean and standard deviation.

Experiment	Conditions	Configurations
Effect of RTT	20 ms, 100 ms, 200 ms RTT; 0% loss	C0, C1, C2, C3
Effect of packet loss	0%, 2%, 5% loss; 50 ms RTT	C0, C1, C2, C3
UDP vs TCP	0%, 2%, 5% loss; 50 ms RTT	C3, T0
Jitter (optional)	0, 20, 50 ms jitter; 100 ms RTT	C2, C3

The main metrics will be perceived input latency, position error, reconciliation correction rate and size, snapshot jitter/rate, stall events, and bandwidth per player.

Expected Outcomes

- Client-side prediction is expected to reduce perceived input latency as RTT increases.
- Interpolation is expected to make remote entity movement smoother under delay and packet loss.
- Input redundancy is expected to reduce corrections under packet loss at a modest bandwidth cost.
- Delta compression is expected to offset part of the additional bandwidth introduced by redundancy.
- The TCP baseline is expected to exhibit delivery stalls under packet loss because of head-of-line blocking.

These are hypotheses to be tested; results that contradict them will still constitute valid findings.

3. Plan of Work – Next 5 Weeks

The first five weeks will focus on completing the core Priority 1 deliverables. Stretch features such as lag-compensated hit detection and multi-player scaling will only be attempted if the core work is completed early.

Week	Planned Work
Week 1	Build the network namespace topology, configure R1/R2 routing, verify connectivity and traceroute, and establish UDP communication across both routers.
Week 2	Implement the authoritative server, basic client, C0 naive configuration, custom UDP message structure and metric logging.
Week 3	Implement C1 prediction and reconciliation, C2 interpolation, and C3 input redundancy and delta-compressed snapshots.
Week 4	Implement the TCP baseline, automate E1–E3 experiments, apply controlled network impairments and collect packet captures.
Week 5	Analyse collected data, generate plots and experiment tables, complete the technical report and README, and rehearse the final demonstration.

Team Split and Individual Contributions

Because the group contains three members, the functional implementation is divided into two members on one side and one member on the other. The split below lists only functional features, as required. Network topology, routing, packet capture, UI, testing and documentation are supporting activities rather than functional features.

Team A – Hatim Mustafa and Fuzail Raza

- Fuzail Raza: Implement the authoritative game server and the custom UDP protocol, including JOIN/JOIN_ACK, INPUT, SNAPSHOT, EVENT and LEAVE messages.
- Fuzail Raza: Implement server-side state management, sequence/acknowledgement handling and delta-compressed snapshots.
- Fuzail Raza: Implement the TCP baseline version of the same game.
- Hatim Mustafa: Implement the network-controlled gameplay environment needed to exercise the functional UDP game across the routed topology and automate the network conditions used by the game.
- Hatim Mustafa: Integrate the protocol/game implementation with the controlled delay, jitter and loss conditions so the functional configurations can be evaluated consistently.

Team B – Zaid Ahmed

- Zaid Ahmed: Implement the pygame multiplayer client and player movement/input handling.
- Zaid Ahmed: Implement C0, the naive client behaviour in which the client renders server-returned state.
- Zaid Ahmed: Implement C1 client-side prediction and reconciliation using replay of unacknowledged inputs.
- Zaid Ahmed: Implement C2 entity interpolation for remote players.
- Zaid Ahmed: Implement C3 client-side input redundancy and integration with the delta-compressed snapshots.
- Zaid Ahmed: Implement headless bot clients and client-side metric logging required for repeatable functional evaluation.

Supporting Work Shared by the Group

- Configure and verify the routed topology, routing tables and traceroute.
- Collect and annotate tcpdump/Wireshark packet captures, including protocol exchanges and TCP comparison traffic.
- Run the controlled experiments and calculate mean and standard deviation for the required metrics.
- Prepare the architecture diagram, technical report, README, demo and AI-use disclosure.

Priority 2 Stretch Work

Only after all Priority 1 milestones are complete:

- Lag-compensated hit detection with server-side input validation.
- Scaling to 4, 8 and 16 players using headless bots.
- Area-of-interest filtering so clients receive only nearby entities.
- Measurement of hit-registration accuracy, bandwidth growth and server CPU/tick performance.

4. References

- IETF RFC 768 – User Datagram Protocol (UDP).
- IETF RFC 9293 – Transmission Control Protocol (TCP).
- Wireshark Documentation – Wireshark Network Protocol Analyzer and packet capture analysis.
- Linux tc-netem Documentation – Network emulation for delay, loss, duplication, corruption and reordering.
- Python Documentation – asyncio networking and asynchronous programming.
- Pygame Documentation – Python game development library used for the project client.